Lab 06 — Gegevens en volumes

Theorie — waarom alles wat een container schrijft samen met de container verdwijnt, de drie manieren om hem een schijf te geven die dat overleeft, en wat rootless verandert aan de permissies.

Doelstellingen

- Begrijpen waarom de *writable layer* van een container geen opslagplaats is.
- **Named volume**, **bind mount** en **tmpfs** uit elkaar houden en weten wanneer je welke kiest.
- `-v` en `--mount` kunnen lezen en schrijven, inclusief `:ro`, `:U` en `:Z`.
- Uitleggen van wie de bestanden in een volume zijn in rootless-modus, en waarom `ls` "Permission denied" zegt.
- De gegevens van een gecontaineriseerde PostgreSQL op twee manieren back-uppen en terugzetten.

---

1. De writable layer is kladpapier

In lab 02 zag je dat een image een stapel read-only lagen is. Wanneer een container start, legt de engine daar een **writable** (beschrijfbare) laag bovenop: leeg, en eigen aan die ene container. Alles wat het proces schrijft — nieuwe bestanden, gewijzigde bestanden, "verwijderde" bestanden — komt in die laag terecht. `podman diff container` somt ze op.

> **→ LINUX**
> Het stapelen gebeurt door **overlayfs**, een bestandssysteem van de kernel. Het toont één samengevoegde weergave van meerdere mappen: de *lowerdirs* (de lagen van de image) en één *upperdir* (de writable layer). Een bestand schrijven maakt het aan in de upperdir. Een bestand uit een lagere laag wijzigen **kopieert** het eerst naar boven (*copy-up*) en wijzigt het daar. Een bestand verwijderen plaatst een *whiteout*, een markering die het verbergt. De image zelf wordt nooit aangeraakt.

Voor gegevens die ertoe doen heeft die laag drie gebreken. Ze **sterft met de container**: `podman rm` wist ze, en een container wordt vaker vervangen dan je denkt (nieuwe image, andere parameter, herstart door een orchestrator). Ze is **traag** bij intensief schrijven, door de copy-up en de omweg via overlay. En ze is **onzichtbaar** voor andere containers: delen is onmogelijk.

> **ONTHOUDEN**
> Een container is **wegwerpbaar** van ontwerp. Moet een gegeven de container overleven, dan hoort het niet *in* de container. Dat principe stuurt dit hele lab — en later de scheiding tussen Pods en PersistentVolumes in Kubernetes.

2. Drie manieren om een schijf te mounten

De engine kan een locatie buiten de container **mounten** op een pad in de container. Wat op dat pad geschreven wordt, gaat niet via de writable layer. Er bestaan drie soorten:

Soort	Waar de gegevens staan	Wie beheert	Wanneer
Named volume	In de opslag van de engine (`~/.local/share/containers/storage/volumes/`)	De engine: `podman volume …`	Gegevens van een database, een cache, een wachtrij: alles wat moet blijven zonder dat jij hoeft te weten *waar*
Bind mount	Gelijk welke map of bestand op de host, door jou gekozen	Jij	Broncode tijdens de ontwikkeling, een configuratiebestand, een uitvoermap
tmpfs	In het werkgeheugen, nooit op schijf	De kernel	Tijdelijke bestanden, ontsleutelde secrets, wegwerpcaches

Twee syntaxen beschrijven ze. De korte, `-v bron:bestemming[:opties]`, is het meest verspreid; de lange, `--mount type=…,source=…,destination=…`, leest makkelijker en laat minder fouten stilzwijgend passeren:

```
podman run -v pgdata:/var/lib/postgresql/data …                    # named volume
podman run -v ./html:/usr/share/nginx/html:ro …                    # bind mount, alleen-lezen
podman run --tmpfs /scratch:size=64m …                             # tmpfs
podman run --mount type=volume,source=pgdata,destination=/var/lib/postgresql/data …
podman run --mount type=bind,source=./html,destination=/usr/share/nginx/html,ro …
```

De regel die ze uit elkaar houdt: een bron die begint met `/`, `./` of `~` is een hostpad, dus een **bind mount**; een gewone naam is een **volume**. Vergeet de `./` voor `html` en je maakt een leeg named volume dat `html` heet — een van de vaakst gemaakte fouten.

3. Named volumes

Een named volume is een map die de engine aanmaakt, een naam geeft en bewaart tot jij ze verwijdert. `podman volume create pgdata` maakt ze aan; `podman run -v pgdata:…` mount ze (en maakt ze aan als ze nog niet bestaat); `podman volume inspect pgdata` toont het `Mountpoint` op de host; `podman volume rm pgdata` vernietigt ze — en weigert zolang een container ze gebruikt.

Bij de **eerste** mount op een map die in de image niet leeg is, **kopieert** de engine de inhoud van de image naar het lege volume (`NeedsCopyUp: true` in de inspect). Daardoor kun je een volume op `/etc/nginx` mounten zonder een lege map te krijgen. Bij latere mounts kopieert hij niets meer.

VALKUIL

Veel images declareren `VOLUME /var/lib/postgresql/data` in hun Dockerfile. Start je `postgres` **zonder** `-v`, dan maakt de engine toch een volume aan, **anoniem**, met een hash van 64 tekens als naam. `podman rm` van de container verwijdert het niet (`rm -v` wel). Na enkele weken toont `podman volume ls` tientallen verweesde volumes en weet niemand nog welk wat bevat. Geef je volumes altijd een naam.

→ POSTGRESQL
De officiële image voert bij het starten `initdb` uit **als en alleen als** de map `PGDATA` (`/var/lib/postgresql/data`) leeg is: dan maakt ze het cluster aan, de superuser `postgres` met het wachtwoord uit `POSTGRES_PASSWORD`, en voert ze de scripts uit `/docker-entrypoint-initdb.d/` uit. Staan er al gegevens, dan hergebruikt ze die zonder iets te initialiseren — en negeert ze `POSTGRES_PASSWORD`. Een persistent volume verandert dus het gedrag van de container bij de tweede start.

4. Bind mounts

Een bind mount maakt een pad van de host zichtbaar in de container, **zonder kopie**: het is dezelfde map, gezien van twee kanten. Wijzig `html/index.html` op de host en nginx serveert meteen de nieuwe versie; schrijf vanuit de container en er verschijnt een bestand in jouw map. Het is het werktuig van de ontwikkeling (herladen zonder rebuild) en van de configuratie (`-v ./nginx.conf:/etc/nginx/nginx.conf:ro`).

De optie `:ro` verbiedt schrijven vanuit de container: een poging faalt met `Read-only file system`. Zet ze standaard, en haal ze alleen weg wanneer de container echt moet schrijven.

→ BEVEILIGING
Op Red Hat, Fedora en CentOS labelt **SELinux** elk bestand en mag een container niets lezen dat niet het label `container_file_t` draagt. Een bind mount faalt er met `Permission denied`, zelfs als root, zolang je geen `:z` (gedeeld tussen containers) of `:Z` (privé voor deze container) toevoegt; die opties labelen de map opnieuw. Ubuntu (dus ook WSL) gebruikt AppArmor, geen SELinux: daar wordt `:Z` aanvaard en gebeurt er niets. Maak er toch een gewoonte van: je commando's werken dan ook op de RHEL-servers van het bedrijf.

→ WINDOWS / WSL
Je Windows-bestanden zijn zichtbaar onder `/mnt/c/…`, maar die mount (`drvfs`, protocol 9p) is **traag** en kent geen Linux-permissies: een bind mount vanaf `/mnt/c` werkt, en verrast daarna met `chmod`-commando's zonder effect en lange toegangstijden. Hou de bestanden van de labs in het Linux-bestandssysteem (`~/labo-docker`) en open ze vanuit Windows via `\\wsl$\Ubuntu\home\…` als dat nodig is.

5. Rootless: van wie zijn de bestanden?

In lab 01 zag je dat de `root` van een rootless container je eigen gebruiker is. De rest van de tabel volgt daaruit: UID 1 van de container komt overeen met het eerste UID van je bereik in `/etc/subuid`, UID 2 met het tweede, enzovoort.

→ LINUX
Een **user namespace** geeft een proces zijn eigen UID-tabel. De kernel vertaalt elk UID van de container naar een echt UID op de host volgens `/proc/<pid>/uid_map`, drie kolommen: *begin container*, *begin host*, *lengte*. Op een standaard Ubuntu/WSL toont `podman unshare cat /proc/self/uid_map` eerst `0 1000 1` (jouw UID wordt root) en dan `1 100000 65536` (de rest van de tabel wijst naar het subuid-bereik). De gebruiker `postgres` in de image, UID 70, schrijft zijn bestanden dus met host-UID **100069**.

Heel concreet gevolg: de bestanden in het PostgreSQL-volume zijn, gezien vanaf de host, van UID 100069, en je `ls` op het `Mountpoint` antwoordt `Permission denied`. Dat is geen defect: de isolatie doet precies wat ze moet doen. Wil je erbij, stap dan de namespace binnen: `podman unshare ls -ln …` toont dan `70 70`, net als in de container.

Het omgekeerde probleem treft bind mounts: jouw map is van UID 1000, en de container ziet dat als **root**. Een proces dat in de container als `--user 1000` draait (host-UID 100999) heeft er **geen enkel recht** op. Drie remedies, van de eenvoudigste naar de netste:

Remedie	Effect	Wanneer
`:U` op de mount	De engine doet een recursieve `chown` naar het UID van de container	Map die alleen voor die container dient
`--userns=keep-id`	Jouw UID houdt hetzelfde nummer in de container	Ontwikkeltools die in je broncode schrijven
`podman unshare chown`	Jij wijzigt de eigenaar "zoals de container hem ziet"	Achteraf herstellen

PODMAN
Docker in klassieke modus heeft dit probleem niet: de daemon is root, en een bestand dat UID 70 in de container schrijft, is *echt* van UID 70 op de host — wat dan weer een beveiligingsprobleem is. `podman unshare`, `:U` en `--userns=keep-id` zijn eigen aan Podman. En `podman volume export` / `podman volume import`, die een volume zonder container naar een tar archiveren, bestaan niet in Docker.

6. tmpfs: een schijf in het geheugen

`--tmpfs /scratch:size=64m` mount een bestandssysteem in RAM: snel, beperkt in grootte, gewist bij het stoppen van de container, en nooit naar de schijf van de host geschreven. Dat is de plaats voor een groot tijdelijk bestand, of voor een ontsleuteld secret dat geen spoor mag nalaten. Let wel op de teller: het geheugen van een tmpfs telt mee in de limiet `--memory` van de container (lab 10).

7. Back-uppen en terugzetten

Twee families, die elkaar niet vervangen:

- **Logische back-up**: je vraagt de software om haar gegevens in haar eigen formaat te exporteren. Voor PostgreSQL maakt `podman exec db pg_dump -U postgres > dump.sql` herspeelbare SQL, **consistent** zelfs terwijl de database draait, en overdraagbaar tussen versies. Dit is de referentieback-up.
- **Fysieke back-up**: je archiveert de bestanden van het volume. `podman volume export pgdata -o pgdata.tar` (Podman), of de overdraagbare methode `podman run --rm -v pgdata:/data:ro -v ./:/backup alpine tar czf /backup/pgdata.tgz -C /data .`. Snel en exact, maar **de container moet gestopt zijn**: de bestanden van een database kopiëren terwijl ze schrijft, levert een inconsistent archief op, soms onherstelbaar.

8. In het bedrijf

- **De database** leeft in een named volume op de ontwikkelmachine en in CI; in productie staat ze meestal **buiten de container** (beheerde dienst) of op netwerkopslag die de orchestrator aan de node koppelt (PersistentVolume, lab 15). Nooit in een bind mount op de schijf van een applicatieserver.
- **De configuratie** (`application.yaml`, `nginx.conf`, certificaten) komt binnen via een alleen-lezen bind mount — of, in lab 08, via omgevingsvariabelen en secrets.

- **De logs** gaan niet naar een bestand in de container, maar naar de standaarduitvoer, waar de engine ze verzamelt (lab 10). Een `logs/`-map in een volume is een teken van een slecht aangepaste applicatie.
- **Uploads** van gebruikers (foto's, PDF's) gaan naar objectopslag (S3, MinIO), niet naar een lokaal volume: een tweede replica zou ze niet zien.

Onthouden

- De writable layer sterft met de container; een gegeven dat moet blijven, zit niet *in* de container.
- Named volume voor de gegevens van de engine, bind mount voor jouw bestanden, tmpfs voor het geheugen.
- Een bron die begint met `/`, `./` of `~` is een bind mount; een gewone naam is een volume — `./` vergeten = leeg volume.
- `VOLUME` in een image maakt anonieme volumes: geef de jouwe een naam en verwijder met `rm -v`.
- `:ro` als standaard; `:Z` voor SELinux; `:U` of `--userns=keep-id` wanneer het UID van de container de map niet bezit.
- In rootless is UID 70 van de container UID 100069 op de host: `podman unshare` om de bestanden te zien.
- `pg_dump` voor een consistente back-up terwijl de database draait; `volume export` of `tar` alleen met gestopte container.

Woordenschat

writable layer: overlay-laag eigen aan de container, vernietigd samen met hem. — **named volume**: map beheerd door de engine, persistent. — **anoniem volume**: volume aangemaakt door `VOLUME`, zonder naam. — **bind mount**: map van de host gemount in de container. — **tmpfs**: bestandssysteem in het geheugen. — **copy-up**: kopie van de inhoud van de image naar een leeg volume bij de eerste mount. — **user namespace / subuid**: vertaaltabel van UID's in rootless. — **`podman unshare`**: een commando uitvoeren in de user namespace van Podman. — **logische / fysieke back-up**: export door de software / kopie van de bestanden.